UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas
UNIDAD DE POSGRADO

Práctica Dirigida N.° 3 — LeNet-5 y AlexNet: de las CNN clásicas a la era moderna

Modalidad de trabajo

Práctica individual o en parejas, para desarrollarse durante la sesión de clase con computadora (Google Colab con GPU recomendado). Duración estimada: 1 hora 15 minutos (Parte 1: 20 min · Parte 2: 50 min · Cierre: 5 min). Ajusten los tiempos si la sesión tiene otra duración.

I. Objetivos de la práctica

- Aplicar la aritmética de convoluciones y pooling (tamaño de salida y conteo de parámetros) al caso de AlexNet.
- Comparar cuantitativamente LeNet-5 y AlexNet: profundidad, número de parámetros, y dónde se concentran (capas convolucionales vs. totalmente conectadas).
- Implementar una versión de AlexNet adaptada a imágenes pequeñas (CIFAR-10) y entrenarla en el tiempo de clase, reutilizando el LeNet-5 de la práctica anterior como línea base.
- Relacionar las innovaciones de AlexNet (ReLU, LRN, dropout, overlapping max-pooling, data augmentation) con los problemas concretos que resuelven.

II. Parte 1 — Aritmética y conceptos de AlexNet (20 min)

Trabajen estas preguntas con lápiz y papel antes de tocar el código; usan la misma fórmula de tamaño de salida de una convolución que vimos con LeNet-5:

$$O = piso[\ (W - F + 2P) / S\ ] + 1$$

Preguntas

1. La tabla de AlexNet en el apunte usa un input de 224×224, pero la aritmética solo da 55×55 en Conv1 si el input real es 227×227. Verifiquen ambos cálculos (con W=224 y con W=227, F=11, S=4, P=0) y expliquen la discrepancia con sus propias palabras.
2. Calculen a mano el tamaño de salida de Pool5, dado que Conv5 produce 13×13×256, con kernel 3×3, stride 2 y sin padding. Comparen su resultado con el valor reportado en la tabla del apunte (6×6×256).
3. FC6 recibe la salida aplanada de Pool5 (6×6×256 = 9,216) y tiene 4096 unidades. Calculen el número de parámetros de FC6 usando (n_in + 1) × n_out, y verifiquen que coincide con los 37,748,736 reportados en la tabla.
4. LeNet-5 tiene aproximadamente 60,000 parámetros; AlexNet tiene aproximadamente 62,000,000. ¿Cuántas veces más grande es AlexNet? De los ~62M de parámetros de AlexNet, casi 95% están en las 3 capas totalmente conectadas (FC6+FC7+FC8). ¿Qué problema práctico genera esta concentración de parámetros, y qué arquitecturas posteriores (mencionadas en el apunte) lo abordan directamente?
5. En términos de sesgo-varianza: ¿por qué el dropout se aplicó únicamente a las capas totalmente conectadas (FC6, FC7) y no a las capas convolucionales?

III. Parte 2 — De LeNet-5 a un "Mini-AlexNet" en CIFAR-10 (50 min)

AlexNet original fue diseñado para imágenes de 224×224 y tardó semanas en entrenarse sobre dos GPUs de 2012; no es viable entrenarlo desde cero en una sesión de clase. En su lugar, vamos a comparar dos redes sobre el mismo dataset pequeño (CIFAR-10, imágenes de 32×32×3, 10 clases): un LeNet-5 adaptado (como línea base) y un "Mini-AlexNet" que conserva el mismo patrón de diseño de AlexNet (5 bloques convolucionales + 3 capas densas, ReLU, dropout, max-pooling) pero con canales y dimensiones reducidas para imágenes pequeñas.

Paso 1 — Datos: CIFAR-10

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616))
])

train_data = datasets.CIFAR10("./data", train=True, download=True,
transform=transform)
test_data  = datasets.CIFAR10("./data", train=False, download=True,
transform=transform)

train_loader = DataLoader(train_data, batch_size=128, shuffle=True)
test_loader  = DataLoader(test_data, batch_size=256, shuffle=False)
```

Paso 2 — Línea base: LeNet-5 adaptado a CIFAR-10 (ya resuelto)

Es el mismo LeNet-5 de la clase anterior, cambiando solo los canales de entrada (3 en vez de 1) y el tamaño de aplanado (la imagen es más grande y ya no necesita padding para llegar a 32×32):

```
class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, kernel_size=5, padding=2)   # 32x32 -> 32x32
        self.pool1 = nn.MaxPool2d(2)                             # 32x32 -> 16x16
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)             # 16x16 -> 12x12
        self.pool2 = nn.MaxPool2d(2)                             # 12x12 -> 6x6
        self.fc1 = nn.Linear(16 * 6 * 6, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, num_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool1(self.relu(self.conv1(x)))
        x = self.pool2(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Paso 3 — Mini-AlexNet para CIFAR-10 (a completar)

Conserva la misma estructura que la AlexNet original (5 conv + 3 FC, ReLU, dropout 0.5 en las FC, max-pooling), pero con canales reducidos y kernels 3×3 en todas las capas (en vez de 11×11 y 5×5), porque la imagen de entrada ya es pequeña (32×32) y kernels grandes la reducirían demasiado rápido. Completen los espacios ___ calculando las dimensiones paso a paso con la fórmula de la Parte 1:

```
class MiniAlexNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Bloque 1: 32x32x3 -> ___x___x64
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(2)                    # -> 16x16x64
        # Bloque 2: 16x16x64 -> ___x___x192
        self.conv2 = nn.Conv2d(___, 192, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(2)                    # -> 8x8x192
        # Bloques 3-5: sin pooling entre ellos (igual que Conv3-Conv4-Conv5 en
AlexNet)
        self.conv3 = nn.Conv2d(192, 384, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(384, ___, kernel_size=3, padding=1)   # 384 -> 256
        self.conv5 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
        self.pool5 = nn.MaxPool2d(2)                    # 8x8 -> ___x___
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc6 = nn.Linear(___ * ___ * 256, 1024)     # aplanado tras pool5
        self.fc7 = nn.Linear(1024, 512)
        self.fc8 = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.pool1(self.relu(self.conv1(x)))
        x = self.pool2(self.relu(self.conv2(x)))
        x = self.relu(self.conv3(x))
        x = self.relu(self.conv4(x))
        x = self.pool5(self.relu(self.conv5(x)))
        x = x.view(x.size(0), -1)
        x = self.dropout(self.relu(self.fc6(x)))
        x = self.dropout(self.relu(self.fc7(x)))
        x = self.fc8(x)
        return x
```

Tareas a realizar

1. Completen los espacios en blanco de MiniAlexNet (canales de conv2 y conv4, dimensión espacial tras pool5, y tamaño de entrada de fc6).
2. Entrenen LeNet5 y MiniAlexNet sobre CIFAR-10 durante 5 épocas cada uno (reutilicen train_one_epoch()/evaluate() de prácticas anteriores, o un ciclo equivalente), usando optim.Adam(lr=1e-3) y nn.CrossEntropyLoss().
3. Registren, para cada modelo: número de parámetros entrenables (sum(p.numel() for p in model.parameters())), tiempo aproximado de entrenamiento, y Test Accuracy final.
4. Comparen ambos modelos en una tabla: parámetros, accuracy y (cualitativamente) profundidad. ¿El modelo con más parámetros fue necesariamente el que mejor generalizó en solo 5 épocas?
5. (Opcional, si sobra tiempo) Reemplacen nn.Dropout(0.5) por nn.Dropout(0.2) en MiniAlexNet y observen si cambia el accuracy o la brecha entre train y test accuracy.

IV. Cierre — Puente hacia VGGNet

AlexNet usa filtros grandes (11×11, 5×5) en sus primeras capas. La próxima arquitectura que veremos, VGGNet, hace la pregunta contraria: ¿qué pasa si en vez de filtros grandes usamos solo filtros 3×3, pero muchos más, apilados en redes más profundas? Anoten esta pregunta: la retomaremos con la misma fórmula de aritmética de convoluciones que usaron hoy.

V. Referencias

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. Advances in Neural Information Processing Systems (NeurIPS).

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 86(11), 2278–2324.